SMART PERSONALIZED AI LEARNING PLATFORM

Full System Architecture & Technical Documentation Blueprint

PREPARED FOR:

Abhinav Shingote

TECHNICAL SCOPE:

Authentication & Google OAuth 2.0 Login Flow,
Nodemailer SMTP Transporter & Password Recovery,
YouTube API v3 video matching, multi-key Gemini rotation,
Streak cron-scheduler checks, PostgreSQL data schemas.

PLATFORM STACK & SECURITY:

React 18, Express, Prisma ORM, PostgreSQL,
Bcryptjs (12 rounds) credentials hashing,
Node-cron, Google Accounts, YouTube v3 Data APIs.

Table of Contents

1. Problem Statement & Solution

THE PROBLEM

Traditional digital education platforms rely on uniform, static syllabus configurations. They fail to adapt to a user's immediate background knowledge level, learning speed, and time constraints. In addition, integration of Large Language Model (LLM) APIs into student-facing apps presents severe challenges:

- **API Rate Limiting (429):** Free tier endpoints quickly hit limits under load. If not handled, this freezes loading animations and locks the client app.
- **Lack of Session History:** Standard LLM chat sessions lose tutor context upon page reload, forcing users to repeatedly restate the topic and their goals.
- **Retention Churn:** A lack of progress representation (such as streaks, badges, or ranks) leads to drop-offs in student learning consistency.

THE SOLUTION

We have engineered a highly personal, stateful learning dashboard with: (1) stateful JSON roadmaps; (2) Postgres DB-backed chat messaging history; (3) context-aware cheat sheets; and (4) a gamified streak/ achievement layer. To ensure high availability, a custom LLM scheduler automatically manages multi-key rotation and initiates fast context-aware fallbacks in milliseconds.

2. System Architecture & Tech Stack

The platform is built on an end-to-end decoupled client-server architecture. React manages state, router guards, and interactive graphics, while the Express API interacts with a PostgreSQL database via Prisma ORM. Key components include:

FRONTEND ARCHITECTURE (SPA)

- **Core Framework:** React v18.2.0 for component isolation, local state management, and lifecycle hooks.
- **Routing & Views:** React Router DOM v6.3.0 for client-side routing, protected dashboards, and redirect flows.
- **Visual Polish & Graphs:** Framer Motion v7.2.1 for premium micro-animations; Chart.js and React-Chartjs-2 for user statistics.
- **Styling Design System:** TailwindCSS v3.1.8 with a deep slate/glassmorphism design pattern.

BACKEND API & DATA

- **Server Core:** Express.js v4.19.2 handles routing, request validation, cookie management, and JWT extraction.
- **Database Engine:** PostgreSQL hosted database representing users, active roadmaps, completed nodes, and achievements.
- **ORM Interface:** Prisma client v5.14.0 for connection pooling, type-safe migrations, and cascading deletions.

AI INTEGRATIONS, YOUTUBE & EMAILS

- **LangChain Core:** @langchain/core (v1.1.48) and @langchain/google-genai (v2.1.31) for LLM prompts and message wrappers.
- **API Models:** Primary Model: gemini-2.5-flash / gemini-1.5-flash. Configured with temperature=0.7 and zero retries for instant rotation.
- **YouTube API v3:** Utilizes googleapis endpoints to dynamically match search parameters for video tutorials based on topics.
- **Nodemailer SMTP:** Transporter client configured with secure ports for SMTP servers to trigger password resets and daily streaks reminders.

3. Core Features & Integration Mechanisms

FEATURE 1: STATEFUL AI ROADMAP GENERATOR

Users enter a subject, skill level, and hours/day availability. The backend prompts Gemini to return a structured JSON syllabus. The JSON is persisted in the database. Client renders daily breakdowns, topic lists, progress checkboxes, and tracks the user's completion percentage in real time.

FEATURE 2: PERSISTENT CHATBOT TUTOR

A sidebar chatbot parses current user queries. Unlike stateless designs, messages are stored in PostgreSQL using the ChatMessage model linked to the user's profile. Upon mounting, the client loads past transcripts via GET /api/chat. The prompt injects context about the active roadmap subject to personalize the chat response.

FEATURE 3: YOUTUBE API V3 COMPONENT MATCHING

To support visual learners, each roadmap topic calls the YouTube Search service. The client initiates GET queries to the YouTube API, matches title queries, pulls thumbnail URLs and embed credentials, and compiles a video panel with relevant playlists for immediate playback directly within the application.

FEATURE 4: EMAIL SCHEDULER & STREAK RETENTION

The app runs an automated database scanner built on node-cron. The cron system tracks each user's lastActiveAt parameter. If the user's activity threshold indicates their streak will reset in less than 2 hours (22-23 hours since last activity), the system connects to SMTP and sends an automated warning email encouraging user login.

FEATURE 5: GAMIFICATION ENGINE & LEADERBOARDS

Completing quizzes awards Experience Points (XP). Level-ups are calculated: Level = floor(XP / 500) + 1. Streaks increment if active on successive days, and reset to 1 if a day is missed. Achievements are automatically unlocked and stored. A global leaderboard fetches all registered users sorted by total XP.

4. YouTube API v3 & Gemini Rotation Details

YOUTUBE API V3 INTEGRATION SPECIFICATIONS

For matching video materials, the application queries Google's YouTube v3 Data API. The key is managed via environmental variable: REACT_APP_YOUTUBE_API_KEY. It communicates with two main API endpoints:

- **Search Endpoint:** https://www.googleapis.com/youtube/v3/search?part=snippet&type=video&q={searchQuery} &maxResults=3&key={apiKey}
- **Videos Endpoint:** https://www.googleapis.com/youtube/v3/videos?part=contentDetails&id={videoIds} &key={apiKey}

Search queries are structured as: `Learn [Topic] in [Subject] Course Tutorial`. The API returns a video JSON response mapping videoId, title, snippet, and high-res thumbnails (e.g. img.youtube.com/vi/{id}/maxresdefault.jpg). A fallback mechanism displays offline mock video sets if the YouTube API key is missing or blocked.

GEMINI MULTI-KEY ROTATION MECHANISM

The backend utilises geminiHelper.js to cycle keys when rate limits (HTTP 429) occur. Key settings are specified under GEMINI_API_KEY_1 to GEMINI_API_KEY_5. Setting maxRetries to 0 bypasses LangChain's exponential backoffs, allowing the system to catch exceptions and switch keys immediately.

GEMINI HELPER CODE SNAPSHOT

```
// backend/utils/geminiHelper.js
async function executeWithKeyRotation(fn) {
  const keys = getGeminiKeys(); // Loads env GEMINI_API_KEY_1..5
  let lastError = null;

  for (let i = 0; i < keys.length; i++) {
    try {
      return await fn(keys[i]); // Run using active key
    } catch (err) {
      lastError = err;
      if (i < keys.length - 1) {
        console.warn(`& þ  Key #${i + 1} failed. Rotating to Key #${i + 2}...`);
        continue;
      }
      throw err; // Out of keys
    }
  }
}
```

5. End-to-End OAuth, SMTP & Encryption Flow

GOOGLE OAUTH 2.0 REGISTRATION & LOGIN FLOW

The platform supports Google OAuth for authentication, implemented as follows:

- **Redirection:** GET /api/auth/google redirects client browser to https://accounts.google.com/o/oauth2/v2/auth with scope userinfo.profile, userinfo.email, client_id, and redirect_uri.
- **OAuth Callback:** GET /api/auth/google/callback receives authorization code, sends a POST to https://oauth2.googleapis.com/token to trade for access_token, and requests profile info from https://www.googleapis.com/oauth2/v2/userinfo.
- **User Creation/Binding:** Checks if user email exists. If not, it creates a new account with a random password hashed using bcryptjs. signs a session JWT cookie, and redirects user to dashboard.

CREDENTIAL SECURITY & BCRYPTJS ENCRYPTION

For local accounts, passwords are encrypted on register and validated on login. The project uses bcryptjs with a salt round factor of 12 (BCRYPT_SALT_ROUNDS=12) for secure password hashing. It compares hashes on login using:

```
// Hashing on Register / User creation
const saltRounds = parseInt(process.env.BCRYPT_SALT_ROUNDS, 10) || 12;
const passwordHash = await bcrypt.hash(password, saltRounds);

// Comparison on Login
const isMatch = await bcrypt.compare(password, user.passwordHash);
```

SMTP EMAIL SENDER & PASSWORD RECOVERY

Emails are dispatched using Nodemailer and a configured SMTP server (e.g. Host: smtp.gmail.com, Port: 587/465, Secure: SSL/TLS). For password recovery:

- **Token Generation:** User POSTs /forgot-password. Server creates a JWT signed with user-specific secret (JWT_SECRET + user.passwordHash) expiring in 15 mins.
- **Recovery Link:** Sends email containing /reset-password/[token] link. The user-specific signature prevents token reuse if the password has already been changed.

6. Database Schema & Prisma Data Models

The application schema is modeled in PostgreSQL using Prisma ORM. Primary keys are CUID strings generated on creation. Foreign keys enforce Cascade deletions to maintain database integrity.

TABLE 1: users

- **Primary Key (PK):** id : String [CUID]
- **Columns:** email (String, Unique, Indexed)
passwordHash (String - Bcrypt 12 rounds)
name (String)
level (Int, Default: 1)
totalXP (Int, Default: 0)
currentStreak (Int, Default: 0)
longestStreak (Int, Default: 0)
lastActiveAt (DateTime, Nullable)
createdAt (DateTime, Default: now())
updatedAt (DateTime, Auto-update)
- **Relations:** roadmaps (Roadmap[]), progress (UserProgress[]), chatMessages (ChatMessage[]), achievements (Achievement[])

TABLE 2: roadmaps

- **Primary Key (PK):** id : String [CUID]
- **Foreign Key (FK):** userId : String (References users.id, onDelete: Cascade)
- **Columns:** subject (String)
duration (Int - Total study days)
skillLevel (String - beginner / intermediate / advanced)
hoursPerDay (Float)
modules (Json - AI day-by-day roadmap objects)
isActive (Boolean, Default: true)
createdAt (DateTime, Default: now())
updatedAt (DateTime)
- **Relations:** user (User), progress (UserProgress[])

TABLE 3: user_progress

- **Primary Key (PK):** id : String [CUID]
- **Foreign Key (FK):** userId : String (References users.id, onDelete: Cascade)
roadmapId : String (References roadmaps.id, onDelete: Cascade)
- **Columns:** dayNumber (Int)
completed (Boolean, Default: false)
quizScore (Int, Nullable - Best attempt score)
xpEarned (Int, Default: 0)
completedAt (DateTime, Nullable)
createdAt (DateTime, Default: now())
- **Constraints:** Unique composite index: [userId, roadmapId, dayNumber] - prevents duplicate progress entries per day/roadmap for a user.

7. Database Schema & Scheduler (Part 2)

TABLE 4: chat_messages

- **Primary Key (PK):** id : String [CUID]
- **Foreign Key (FK):** userId : String (References users.id, onDelete: Cascade)
- **Columns:** sender (String - "user" | "bot")
text (String - chat text contents)
timestamp (DateTime, Default: now())
- **Relations:** user (User)

TABLE 5: achievements

- **Primary Key (PK):** id : String [CUID]
- **Foreign Key (FK):** userId : String (References users.id, onDelete: Cascade)
- **Columns:** badgeId (String - trigger keyword: first_quiz, level_5, etc.)
title (String - badge display name)
description (String - badge unlock instructions)
icon (String - Emoji representation)
unlockedAt (DateTime, Default: now())
- **Constraints:** Unique composite index: [userId, badgeId] - guarantees a user can only unlock each badge once.

HOURLY STREAK SCANNER CRON SCHEMA

A background worker in services/scheduler.js checks user activity patterns hourly (cron: 0 * * * *). It queries users at risk of losing active streaks:

```
// Cron hourly streak validation (services/scheduler.js)
const minDate = new Date(now.getTime() - 23 * 60 * 60 * 1000); // 23h ago
const maxDate = new Date(now.getTime() - 22 * 60 * 60 * 1000); // 22h ago

const usersAtRisk = await prisma.user.findMany({
  where: {
    currentStreak: { gt: 0 },
    lastActiveAt: {
      gte: minDate,
      lte: maxDate,
    },
  },
});
```

CONCLUSION & EXPORT SUCCESS

The platform represents a robust, highly-available solution. Using multi-key key rotation, SMTP triggers, and Google/YouTube APIs, it delivers a secure learning environment. The database maintains schema validation with composite indices, type-safe queries, and relational integrity.